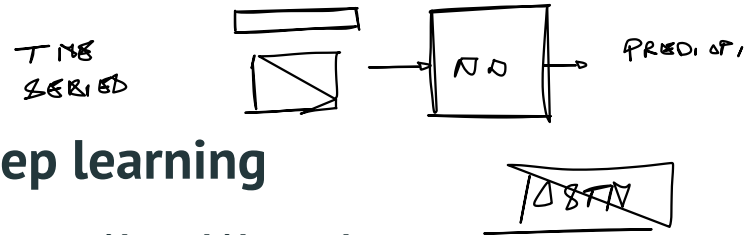


Deep learning

Recurrent Neural Networks

Wouter Gevaert & Marie Dewitte

Overview

Recurrent Neural Networks

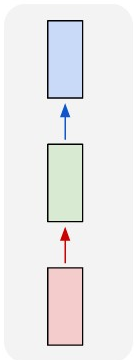
Long Short Term Memory networks (LSTM)

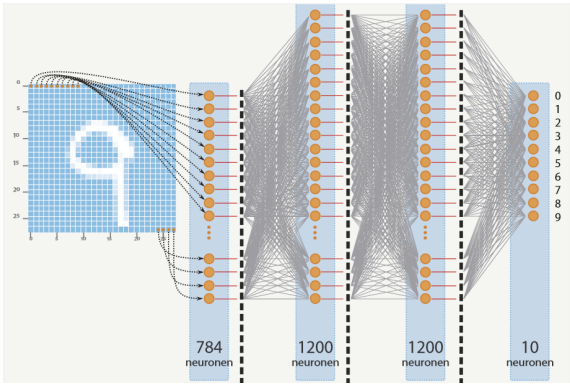
Gated Recurrent Unit (GRU)

Recurrent Neural Networks

RNN modes

Vanilla Neural Network

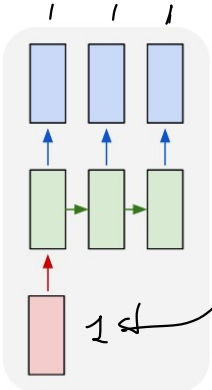




A fixed input is sent through a set of hidden layers and produces a single output.

RNN modes

One to many



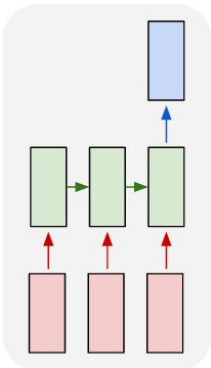
"man in black shirt is playing guitar."

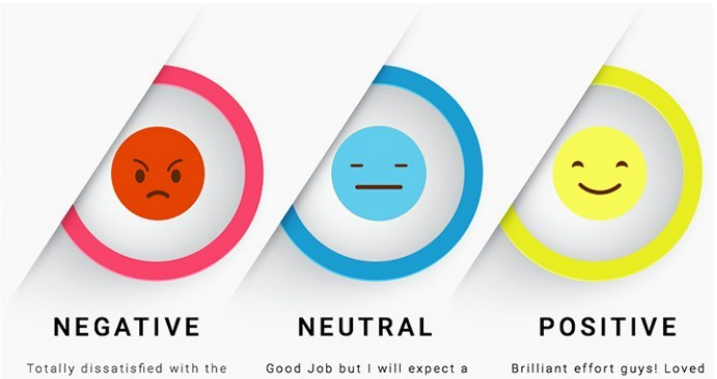
"construction worker in orange safety vest is working on road."

The input length is fixed but the output has a varying length.

RNN modes

Many to one

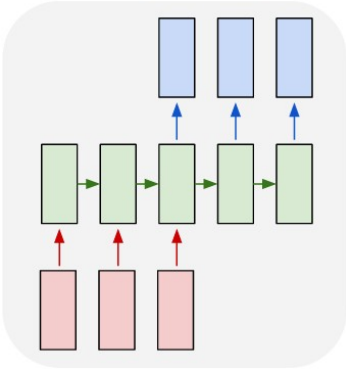


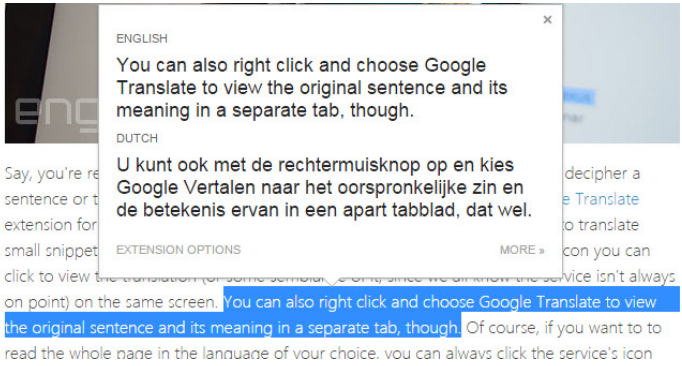


The input has a varying length but the output is fixed .

RNN modes

Many to many - variant 1

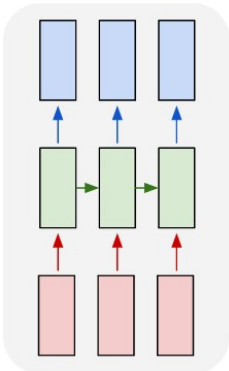


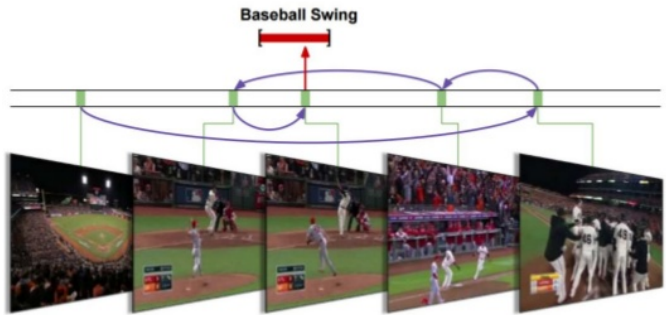


Both the input and the output have a varying length. For example: a sequence of words being converted to another sequence of words.

RNN modes

Many to many - variant 2





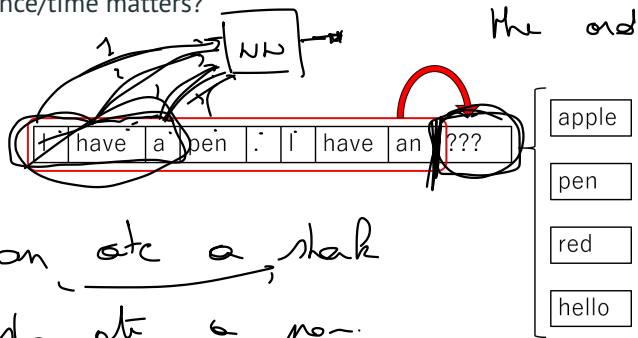
Both the input and the output have a varying length. Make a prediction for each element in the sequence. For example: video classification on the level of video frames.

Applications and examples

TIME SERIES → sequence of values where the order is important

Text prediction

What if the sequence/time matters?



Applications and examples

Text prediction - Shakespeare

VIOLA:
Why, Salisbury must find his flesh and thought
That which I am not aps, not a man and in fire,
To show the reining of the raven and the wars
To grace my hand reproach within, and not a fair are hand,
That Caesar and my goodly father's world;
When I was heaven of presence and our fleets,
We spare with hours, but cut thy council I am great,
Murdered and by thy master's ready there
My power to give thee but so much as hell:
Some service in the noble bondman here,
Would show him to her wine.

KING LEAR:
O, if you were a feeble sight, the courtesy of your law,
Your sight and several breath, will wear the gods
With his heads, and my hands are wonder'd at the deeds,
So drop upon your lordship's head, and your opinion
Shall be against your honour.

PANDARUS:
Alas, I think he shall be come approached and the day
When little srain would be attain'd into being never fed,
And who is but a chain and subjects of his death,
I should not sleep.

Second Senator:
They are away this miseries, produced upon my soul,
Breaking and strongly should be buried, when I perish
The earth and thoughts of many states.

DUKE VINCENTIO:
Well, your wit is in the care of side and that.

Second Lord:
They would be ruled after this chamber, and
my fair nues begun out of the fact, to be conveyed,
Whose noble souls I'll have the heart of the wars.

Bron: `http://karpathy.github.io/2015/05/21/rnn-effectiveness/`

Applications and examples

Text prediction - Latex generation

Proof. Omitted. □

Lemma 0.1. *Let $\mathcal{C}$ be a set of the construction.*

Let $\mathcal{C}$ be a gerber covering. Let $\mathcal{F}$ be a quasi-coherent sheaves of $\mathcal{O}$-modules. We have to show that

$$\mathcal{O}_{\mathcal{O}_X} = \mathcal{O}_X(\mathcal{L})$$

.

Proof. This is an algebraic space with the composition of sheaves $\mathcal{F}$ on $X_{\acute{e}tale}$ we have

$$\mathcal{O}_X(\mathcal{F}) = \{morph_1 \times_{\mathcal{O}_X} (\mathcal{G}, \mathcal{F})\}$$

where $\mathcal{G}$ defines an isomorphism $\mathcal{F} \to \mathcal{F}$ of $\mathcal{O}$-modules. □

Lemma 0.2. *This is an integer $\mathcal{Z}$ is injective.*

Proof. See Spaces, Lemma ??. □

Lemma 0.3. *Let S be a scheme. Let X be a scheme and X is an affine open covering. Let $\mathcal{U} \subset \mathcal{X}$ be a canonical and locally of finite type. Let X be a scheme. Let X be a scheme which is equal to the formal complex.*

The following to the construction of the lemma follows.

Let X be a scheme. Let X be a scheme covering. Let

$$b : X \to Y' \to Y \to Y \to Y' \times_X Y \to X.$$

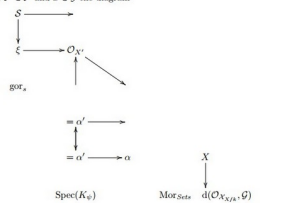
be a morphism of algebraic spaces over S and Y.

Proof. Let X be a nonzero scheme of X. Let X be an algebraic space. Let $\mathcal{F}$ be a quasi-coherent sheaf of $\mathcal{O}_X$-modules. The following are equivalent

(1) $\mathcal{F}$ is an algebraic space over S.
(2) If X is an affine open covering.

Consider a common structure on X and X the functor $\mathcal{O}_X(U)$ which is locally of finite type. □

This since $\mathcal{F} \in \mathcal{F}$ and $x \in \mathcal{G}$ the diagram



is a limit. Then $\mathcal{G}$ is a finite type and assume S is a flat and $\mathcal{F}$ and $\mathcal{G}$ is a finite type f_*. This is of finite type diagrams, and

- the composition of $\mathcal{G}$ is a regular sequence,
- $\mathcal{O}_{X'}$ is a sheaf of rings.

□

Proof. We have see that $X = \mathrm{Spec}(R)$ and $\mathcal{F}$ is a finite type representable by algebraic space. The property $\mathcal{F}$ is a finite morphism of algebraic stacks. Then the cohomology of X is an open neighbourhood of U. □

Proof. This is clear that $\mathcal{G}$ is a finite presentation, see Lemmas ??.
A *reduced above* we conclude that U is an open covering of $\mathcal{C}$. The functor $\mathcal{F}$ is a "field

$$\mathcal{O}_{X,x} \longrightarrow \mathcal{F}_{\overline{x}} \quad \text{-1}(\mathcal{O}_{X_{etale}}) \longrightarrow \mathcal{O}_{X_i}^{-1}\mathcal{O}_{X_i}(\mathcal{O}_{X_i}^{\overline{x}})$$

is an isomorphism of covering of $\mathcal{O}_{X_i}$. If $\mathcal{F}$ is the unique element of $\mathcal{F}$ such that X is an isomorphism.
The property $\mathcal{F}$ is a disjoint union of Proposition ?? and we can filtered set of presentations of a scheme $\mathcal{O}_X$-algebra with $\mathcal{F}$ are opens of finite type over S.
If $\mathcal{F}$ is a scheme theoretic image points. □

If $\mathcal{F}$ is a finite direct sum $\mathcal{O}_{X_i}$ is a closed immersion, see Lemma ??. This is a sequence of $\mathcal{F}$ is a similar morphism.

Applications and examples

Text prediction - Linux source code

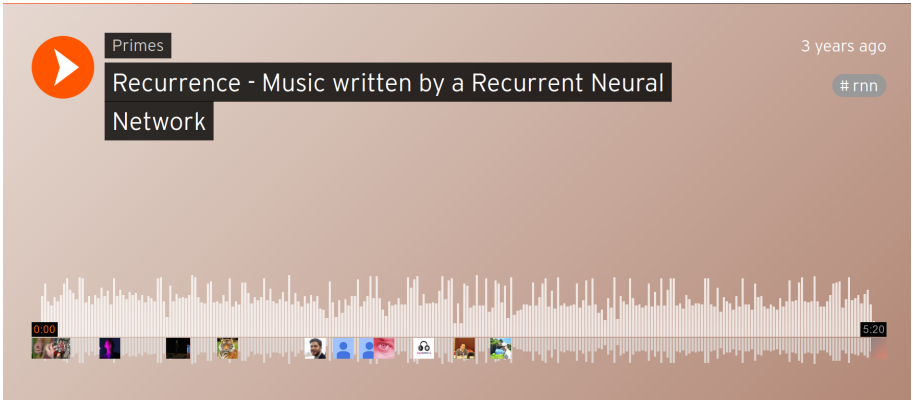
```
/*
 * Increment the size file of the new incorrect UI_FILTER group information
 * of the size generatively.
 */
static int indicate_policy(void)
{
  int error;
  if (fd == MARN_EPT) {
    /*
     * The kernel blank will coeld it to userspace.
     */
    if (ss->segment < mem_total)
      unblock_graph_and_set_blocked();
    else
      ret = 1;
    goto bail;
  }
  segaddr = in_SB(in.addr);
  selector = seg / 16;
  setup_works = true;
  for (i = 0; i < blocks; i++) {
    seq = buf[i++];
    bpf = bd->bd.next + i * search;
    if (fd) {
      current = blocked;
    }
  }
  rw->name = "Getjbbregs";
  bprm_self_clearl(&iv->version);
  regs->new = blocks[(BPF_STATS << info->historidac)] | PFMR_CLOBATHINC_SECONDS << 12;
  return segtable;
}
```

```
/*
 * If this error is set, we will need anything right after that BSD.
 */
static void action_new_function(struct s_stat_info *wb)
{
  unsigned long flags;
  int lel_idx_bit = e->edd, *sys & ~((unsigned long) *FIRST_COMPAT);
  buf[0] = 0xFFFFFFFF & (bit << 4);
  min(inc, slist->bytes);
  printk(KERN_WARNING "Memory allocated %02x/%02x, "
    "original MLL instead\n"),
    min(min(multi_run - s->len, max) * num_data_in),
    frame_pos, sz + first_seg);
  div_u64_w(val, inb_p);
  spin_unlock(&disk->queue_lock);
  mutex_unlock(&s->sock->mutex);
  mutex_unlock(&func->mutex);
  return disassemble(info->pending_bh);
}

static void num_serial_settings(struct tty_struct *tty)
{
  if (tty == tty)
    disable_single_st_p(dev);
  pci_disable_spool(port);
  return 0;
}
```

Applications and examples

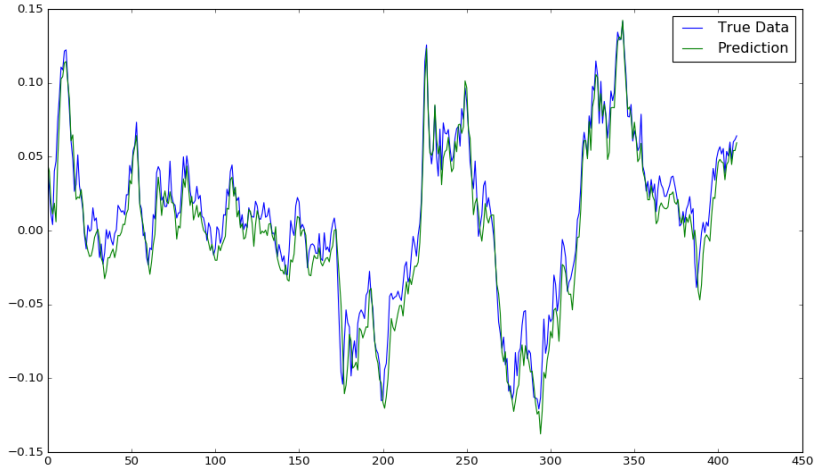
Music generation



https://soundcloud.com/optometrist-prime/recurrence-music-written-by-a-recurrent-neural-network

Applications and examples

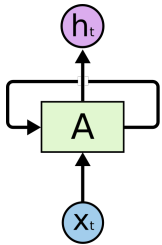
Prediction of time series - Stock market



Architecture of a RNN

Recurrent Neural Networks

RNN cell



=

An RNN reads a sequence word-by-word

Each time step feeds the previous hidden state + new input

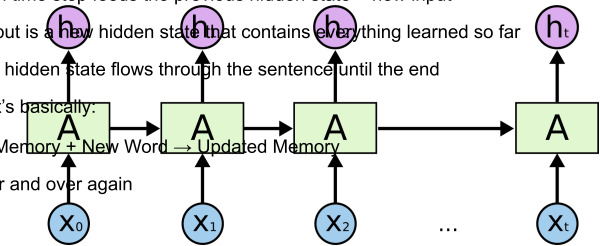
Output is a new hidden state that contains everything learned so far

This hidden state flows through the sentence until the end

So it's basically:

Memory + New Word → Updated Memory

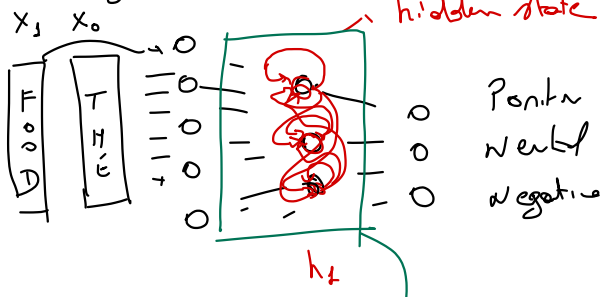
Over and over again



$$h_t = f(h_{t-1}, x_t, \theta)$$

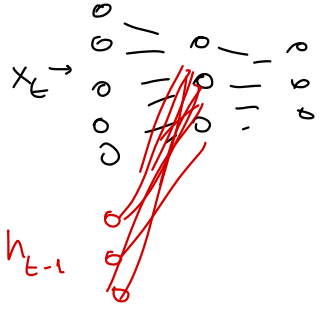
"The food was great"

x_3 x_2 x_1 x_0



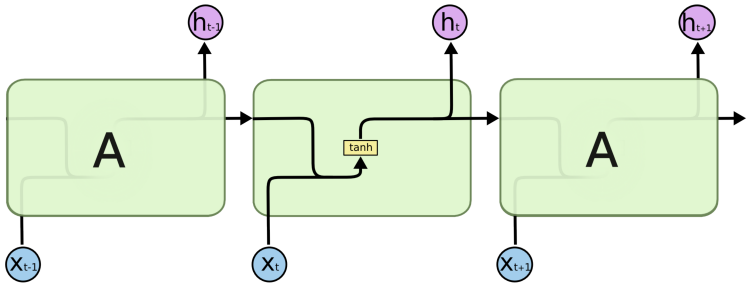
hidden state = "knowledge the NN has about the past"

RNN Cell



Architecture of a RNN

RNN cell



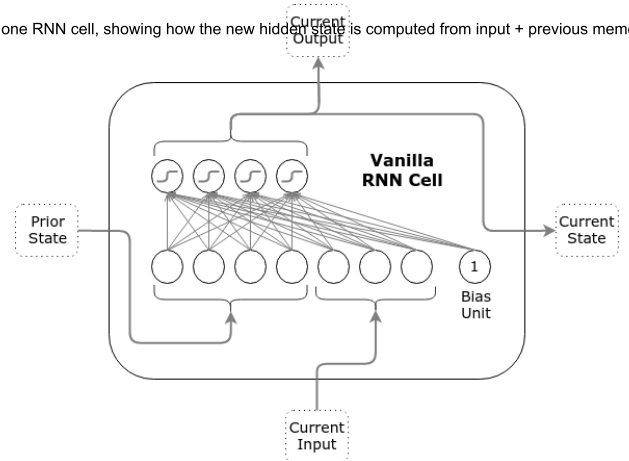
$$h_t = f(h_{t-1}, x_t, \theta)$$

Architecture of a RNN

RNN cell

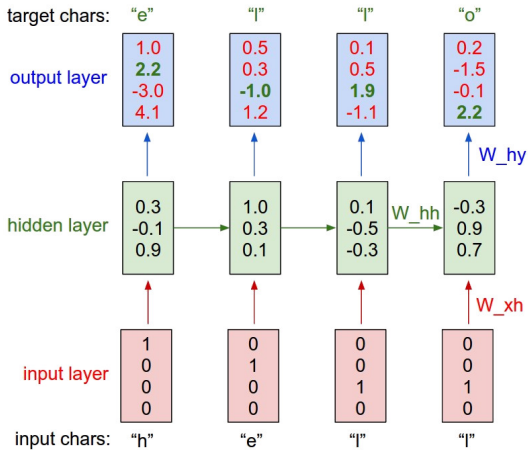
Slide 1:
Shows how the RNN processes a sequence step by step, passing memory forward.

Slide 2:
Zooms into the inside of one RNN cell, showing how the new hidden state is computed from input + previous memory.



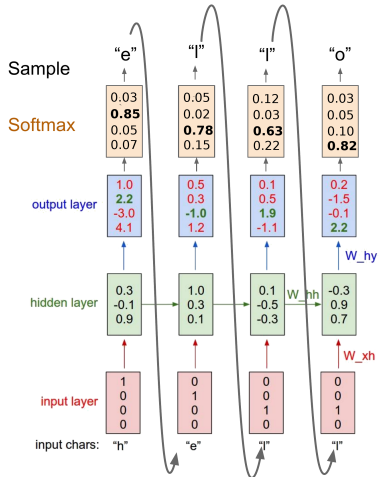
Architecture of a RNN

Example - character prediction - training



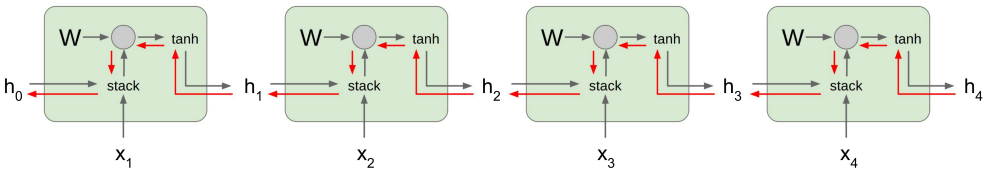
Architecture of a RNN

Example - character prediction - testing



Problems with RNN

Vanishing & exploding gradients



Gradient flow

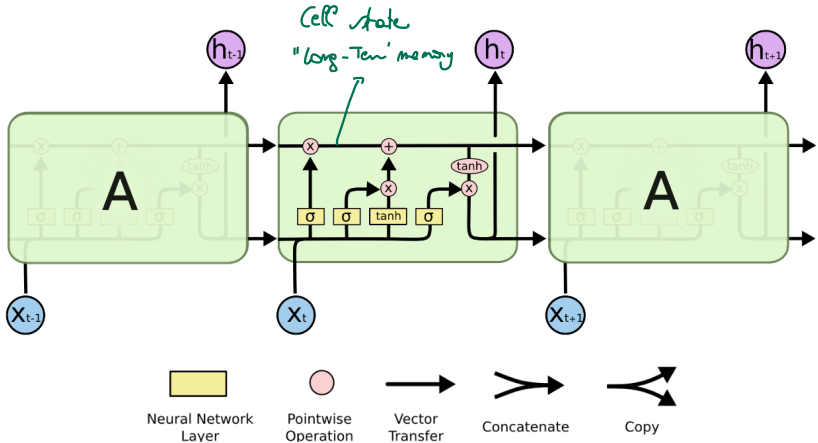
- No long-term memory.
- Vanishing gradient or exploding gradient problem.

Solution: LSTM cell

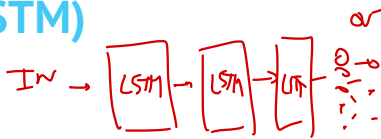
Long Short Term Memory networks (LSTM)

Long Short Term Memory networks (LSTM)

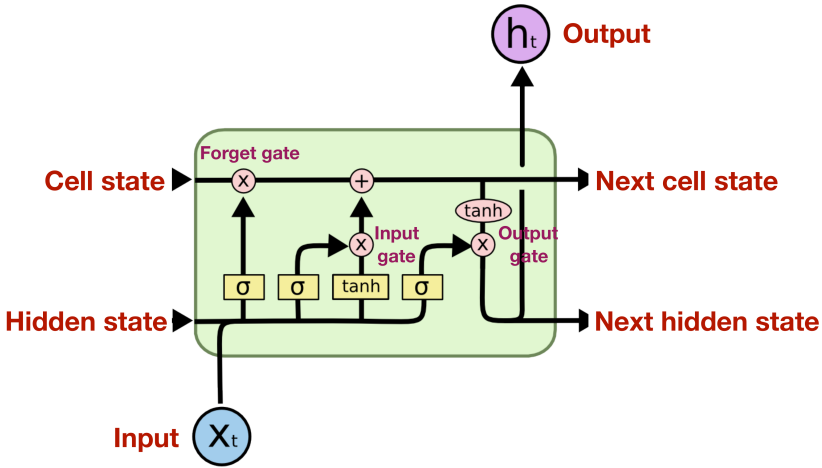
LSTM structure

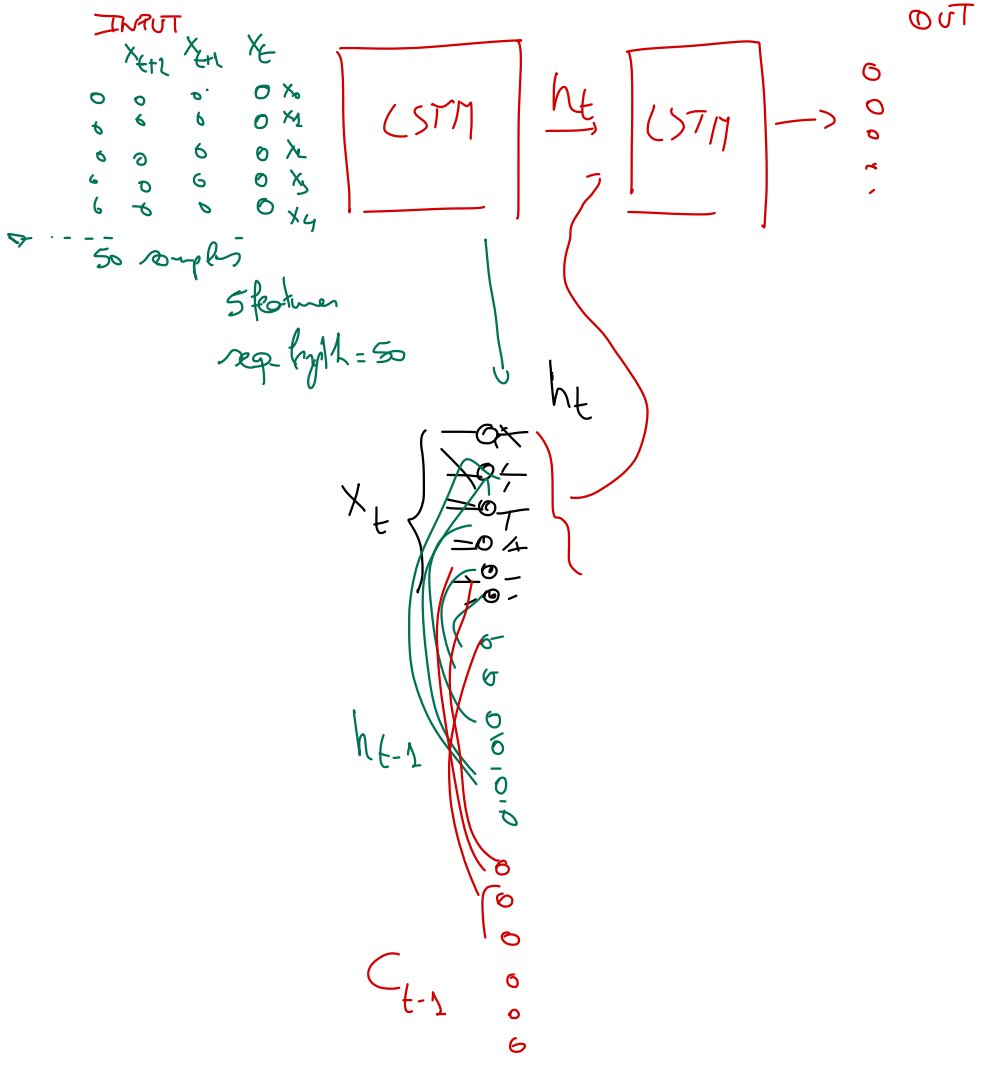


Long Short Term Memory networks (LSTM)



LSTM structure

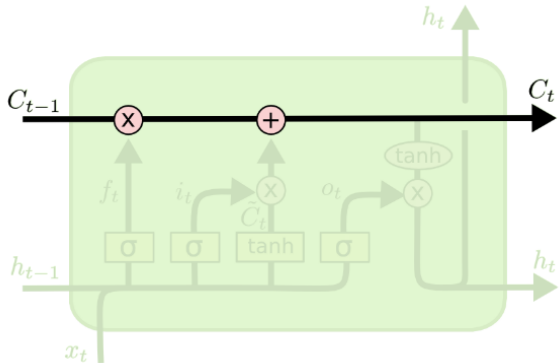



INPUT
OUT
LSTM
LSTM
50 samples
5 features
seq length = 50

Long Short Term Memory networks (LSTM)

Cell state

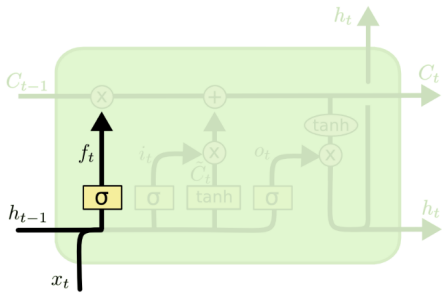
Long-term memory C_t



Long Short Term Memory networks (LSTM)

Forget gate

Determines to what extent the memory has to be erased.

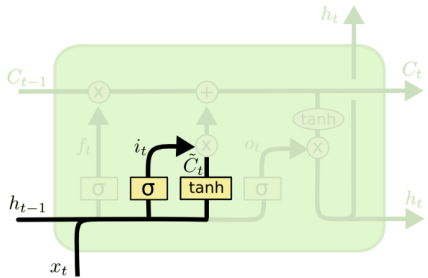


$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Long Short Term Memory networks (LSTM)

Input gate

Determines to what extent new information has to be added to the cell state.



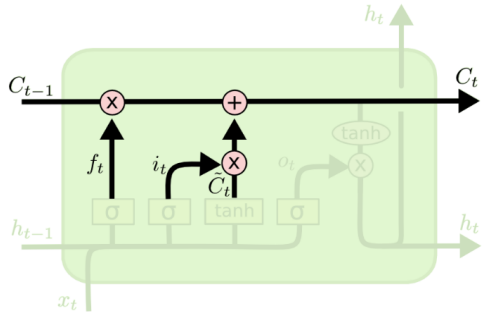
$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Long Short Term Memory networks (LSTM)

Cell state update

Update the 'old' cell state.



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

that control what to remember and what to forget.**

These slides specifically cover:

1. **Input gate**
2. **Cell state update**

Let's decode them.

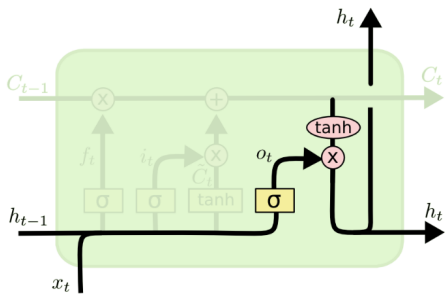
Slide 1 – The **Input Gate** (What new information should enter memory?)

This part controls **how much new

Long Short Term Memory networks (LSTM)

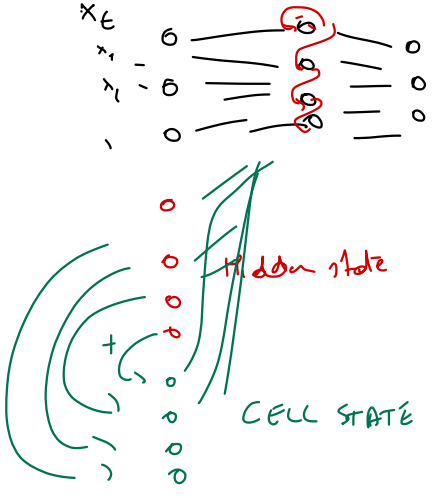
Cell state update

Predict the output h_t.



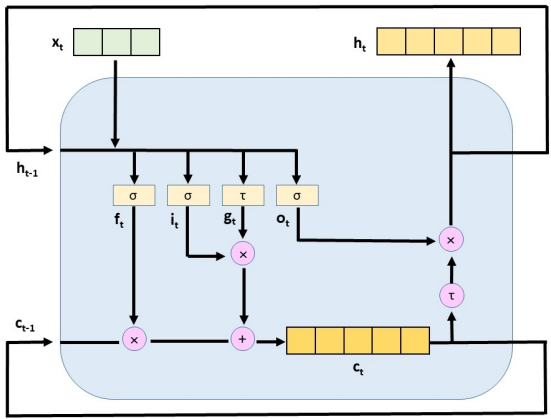
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

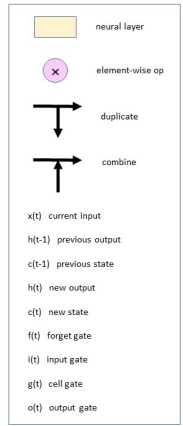
$$h_t = o_t * \tanh\left(C_t\right)$$


Hidden state
CELL STATE

Long Short Term Memory networks (LSTM)

Information flow





$$f_t = \sigma(W_{if}x_t + b_{if} + W_{hf}h_{(t-1)} + b_{hf})$$

$$i_t = \sigma(W_{ii}x_t + b_{ii} + W_{hi}h_{(t-1)} + b_{hi})$$

$$g_t = \tau(W_{ig}x_t + b_{ig} + W_{hg}h_{(t-1)} + b_{hg})$$

$$o_t = \sigma(W_{io}x_t + b_{io} + W_{ho}h_{(t-1)} + b_{ho})$$

$$c_t = f_t \circ c_{(t-1)} + i_t \circ g_t$$

$$h_t = o_t \circ \tau(c_t)$$

Long Short Term Memory networks (LSTM)

Here you go — a short, clean summary you can copy-paste straight into your notes:

LSTM Input Shape – Short Summary

An LSTM does not take a single vector as input. It takes a **sequence of vectors**, one for each time step. Each vector contains several features.

The required input shape for an LSTM is:

[
(\text{batch_size},\ \text{sequence_length},\ \text{n_features})
]

* **batch_size** = number of sequences processed at once
* **sequence_length** = how many time steps (e.g., words, frames, days)
* **n_features** = number of values at each time step

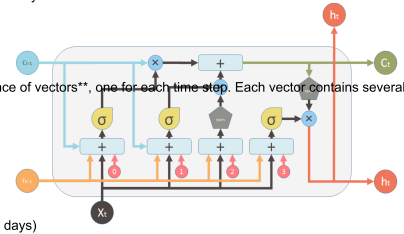
Example:
If each sample has 10 time steps and each step has 5 features, the input becomes:

[
(32,\ 10,\ 5)
]

for a batch size of 32.

In short: **LSTMs read sequences over time, and each time step is a feature vector.**

Input shape

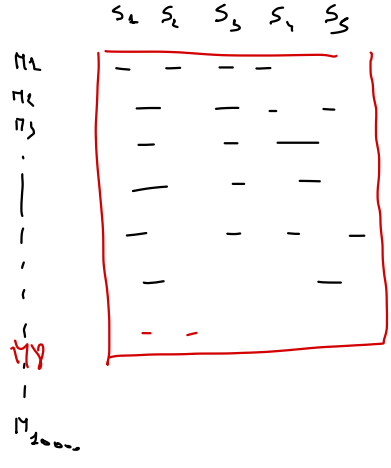


Convolutional NN

dimension of the training set

$X\_train.shape \Rightarrow (1000, 600, 400, 3)$

1000: # samples # images
600: # rows
400: # cols
3: # color channels



X_train.

LSTM

feature

$X\_train.shape \; (1000, 50, 7)$

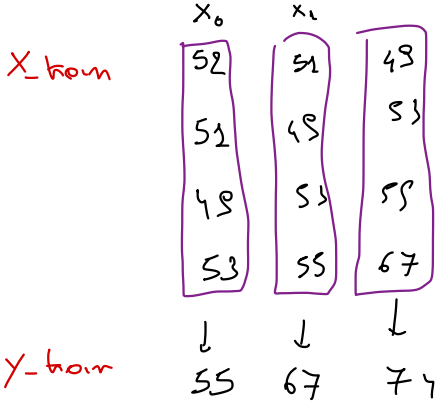
1000: # sequences
50: seq-length

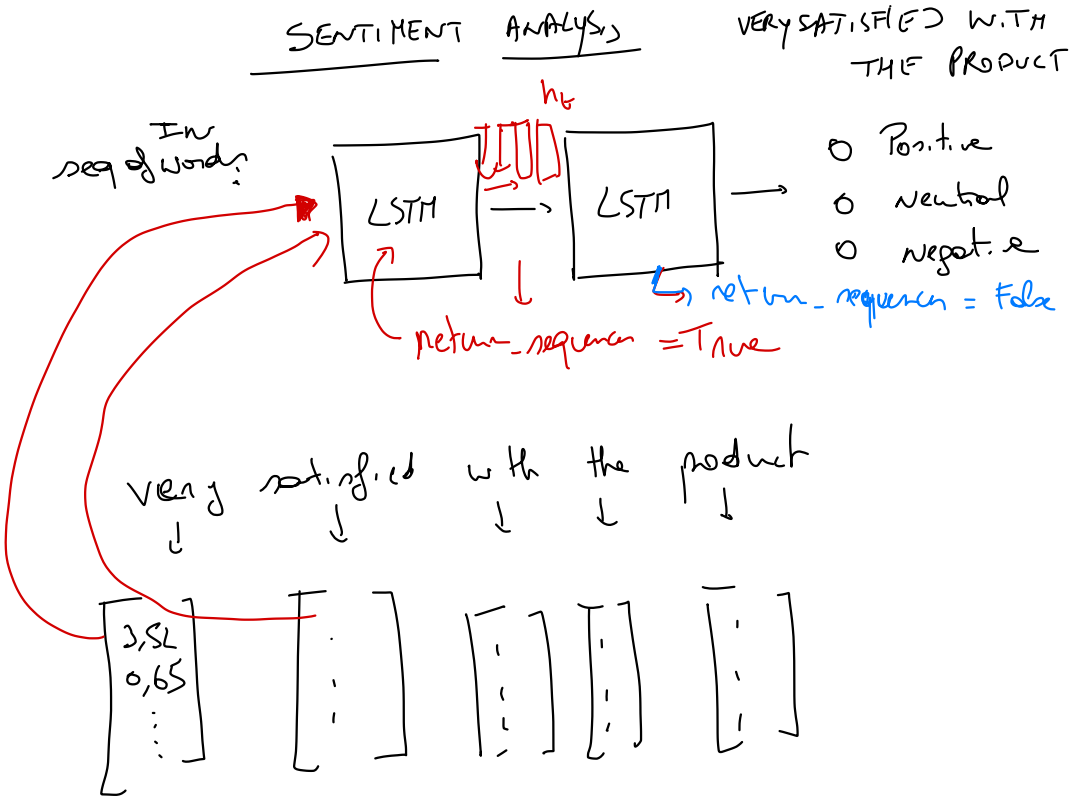
example: - a machine is equiped with 5 sensor
- 1 measure each minute.
- 1 sequence consists of 60 measures

$(1000, 60, 5)$

ex Bitcoin price

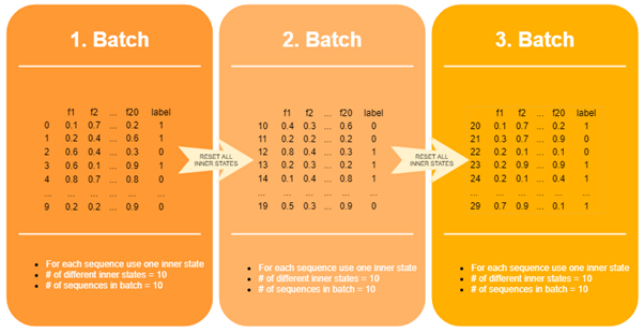
52 | 51 | 48 | 53 | 55 | 67 | 74 . 78 85 . .




SENTIMENT ANALYSIS
VERY SATISFIED WITH THE PRODUCT
In seq of words
LSTM
LSTM
h_t
Positive
Neutral
negative
return_sequences = False
return_sequences = True
very satisfied with the product
3,51
0,65

Long Short Term Memory networks (LSTM)

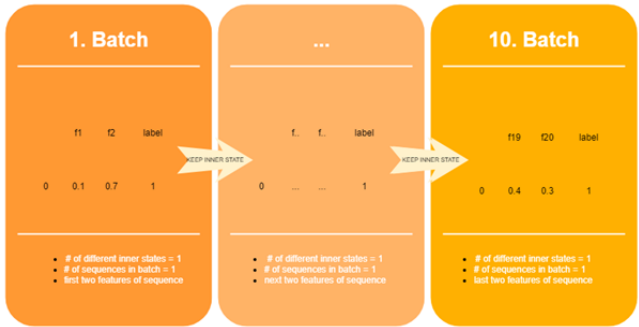
Stateless LSTM



In a stateless LSTM layer, a batch has x (size of batch) inner states, one for each sequence. The inner state and all the outputs of one row / sequence is deleted when the sequence is processed i.e. when one batch is processed.

Long Short Term Memory networks (LSTM)

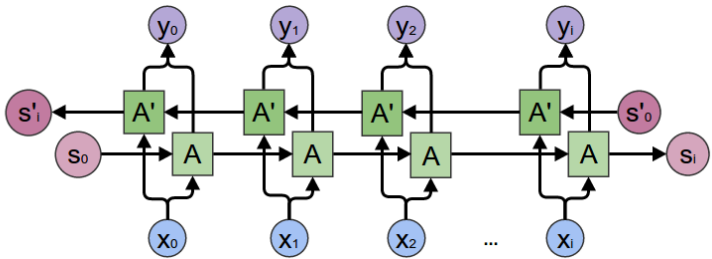
Statefull LSTM



In a stateful LSTM layer we don't reset the inner state and the outputs after each batch. Rather we delete them after each epoch, which literally means that we use and update one internal state for one sequence across multiple batches.

Long Short Term Memory networks (LSTM)

Bidirectional LSTM

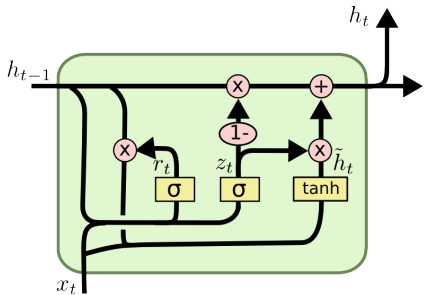


Bidirectional LSTMs train two instead of one LSTMs on the input sequence. The first on the input sequence as-is and the second on a reversed copy of the input sequence. This can provide additional context to the network and result in faster and even fuller learning on the problem.

Gated Recurrent Unit (GRU)

Gated Recurrent Unit

GRU structure



$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

GRU – Short Summary

A **GRU (Gated Recurrent Unit)** is a simplified version of an LSTM.
It uses **two gates instead of three**, making it faster and easier to train while still handling long-term dependencies.

Gates in a GRU:

* **Update gate** (z_t):
  Controls how much of the previous hidden state is kept.
  [
  z_t = \sigma(W_z [h_{t-1}, x_t])
  ]

* **Reset gate** (r_t):
  Controls how much past information to forget.
  [
  r_t = \sigma(W_r [h_{t-1}, x_t])
  ]

* **Candidate hidden state** (\tilde{h}_t):
  New memory created from current input and reset state.
  [
  \tilde{h}*t = \tanh(W [r_t * h_{t-1}, x_t])
  ]

* **Final hidden state** (h_t):
  Mixture of old memory and new candidate.
  [
  h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t
  ]

GRU vs LSTM (quick comparison)

* **LSTM:**
  Has **3 gates** (input, forget, output) and keeps a separate **cell state** (C_t).
  More control → better for very long sequences.

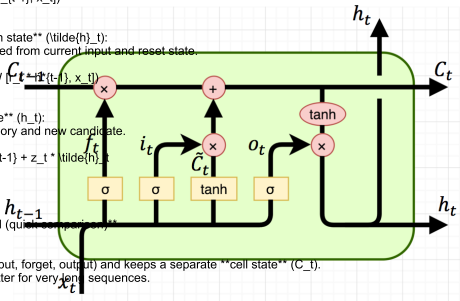
* **GRU:**
  Has **2 gates** (update, reset) and **NO separate cell state**.
  Simpler, faster, fewer parameters.

In short:

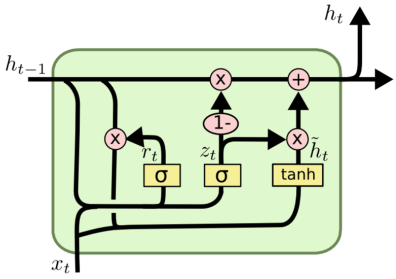
* LSTM = more complex, more memory control.
* GRU = simpler, faster, often performs just as well.

Gated Recurrent Unit

GRU versus LSTM



Long Short-Term Memory

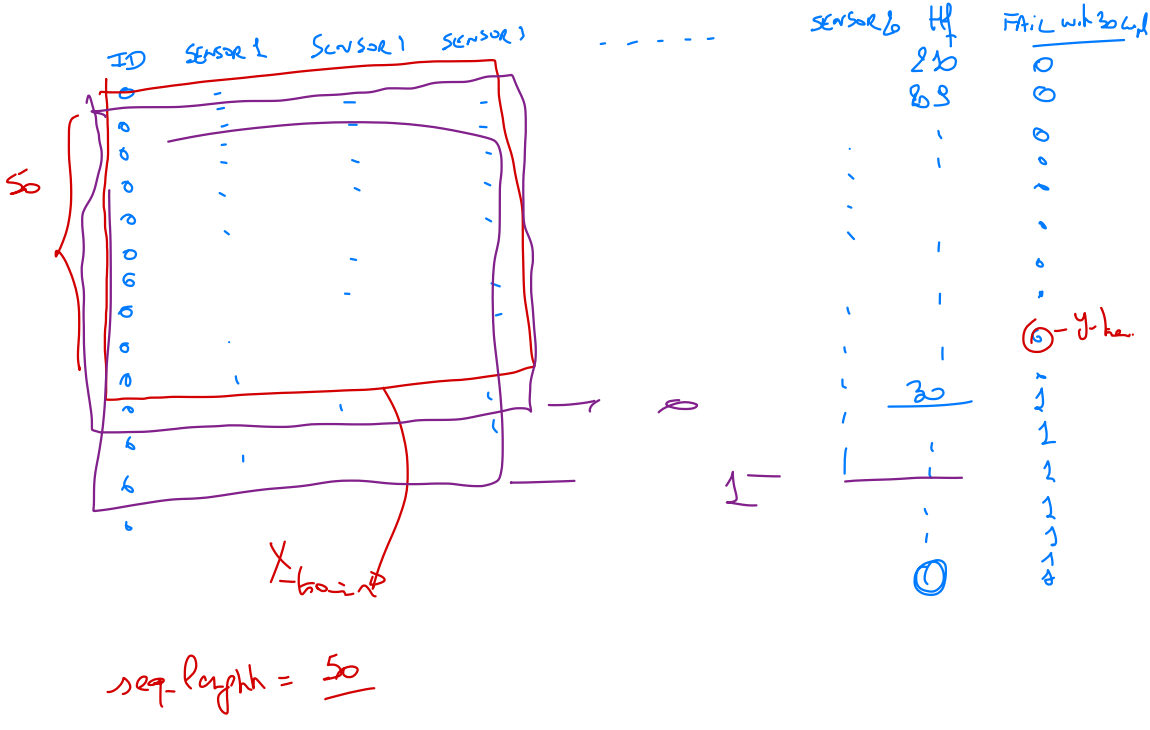


Gated Recurrent Unit

Gated Recurrent Unit

GRU versus LSTM

- GRU does not have a (long-term) memory unit.
- Performance is similar to the performance of LSTM. Especially with small datasets.
- GRU trains faster than LSTM.
- In theory, LSTM has a bigger long-term memory and can therefore memorize longer sequences.



seq_length = 50